

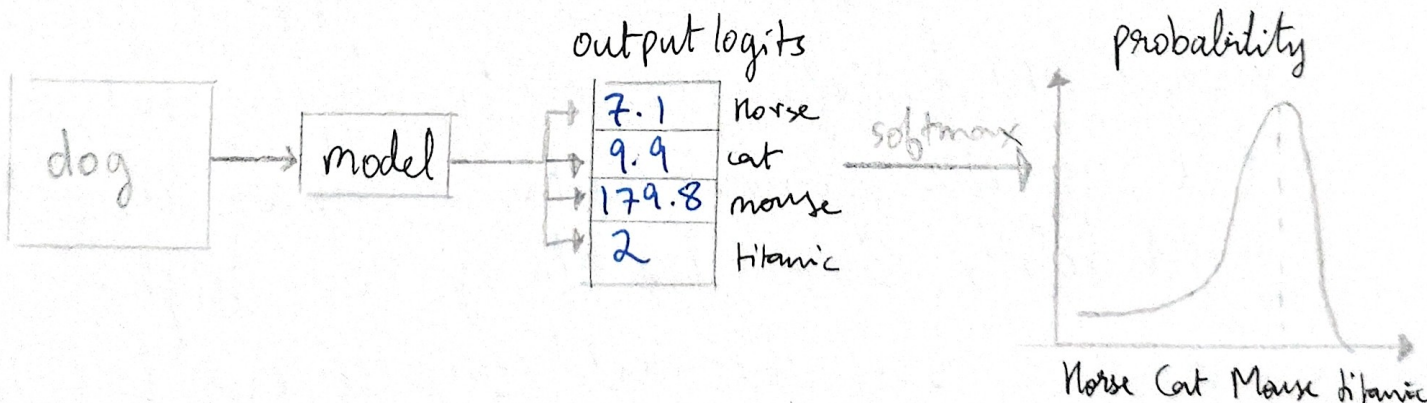
what problem existed before the paper?

Let us first start by defining cross-entropy loss:

$$L_{\text{loss}} = - \sum_{k=1}^K \log(P(k)) \cdot q(k)$$

where $P(k)$ is the model's predicted probability that a certain sample x belongs to class k , $q(k)$ is the ground truth label.

Let us take an example, consider the classification of horse, cat, mouse and Titanic ship images. The model takes in a sample of a ~~dog~~ and outputs the following logits: mouse



The probability distribution we obtain is then

log transformed and multiplied by the ground truth distribution q .

THE PROBLEM:

Consider hard labels where the ground truth distribution is given by $q_k = 1$ if $y = k$ else $q_k = 0$. This induces two problems:

1. Overfitting during model training.
2. Overconfidence during testing.

How? :

Let us consider a training example of a mouse incorrectly labelled as a cat. When fed into the model, due to the model using hard labels, it predicts mouse with a very high probability. (rightly so!)

But as the ground truth is given as cat, hard labels cause loss to go very high.

The only way the model can decrease loss is by overfitting on mislabelled data.

Another example is a clean fact: A model that could give logits as infinite values would actually overfit on all of the training data perfectly. It would give $+\infty$ for logits which is correct and $-\infty$ for other ones.

WHY OVERCONFIDENT PREDICTIONS ARE BAD:

Real life example:

Overconfidence essentially (in a more basic level) means that when a model predicts with 90% probability, it doesn't ~~not~~ mean that the model is right 90% of the time, the real chance is more like 75%.

The model overestimates the probability that it is right.

An example of when this is bad comes from imaging of brain tumors, or something medical, where doctors need to be able to evaluate.

HOW LABEL SMOOTHING WORKS

Let us define the dirac delta $\delta_{k,y}$ as a function that equals 1 for $k=y$ and 0 in other cases.

→ Label smoothing changes our target's ground truth distribution $q(k)$ to:

$$q(k/n) = (1 - \epsilon) \delta_{k,y} + \frac{\epsilon}{K} u(k)$$

With label smoothing, we also see that no logits can balloon up to extremely high values close to ∞ ; this is because:

$$q_k = 1 - \alpha + \frac{\alpha}{K} \quad \text{for true class}$$

That means that the optimal probability of the true label class is $q_k = 1 - \frac{\alpha}{2}$ (at the max case of $K=2$).

HIGH LEVEL INTUITION:

Say we take the original formula

from the original paper on label smoothing:

$$q'(k|x) = (1-\epsilon) \delta_{k,y} + \epsilon u(k)$$

$u(k)$ is a unique constant function which we allow to be $1/k$ to normalize and provide equal weightages to classes for Label smoothing. The new loss becomes:

$$\begin{aligned} \text{Loss}'(q', p) &= - \sum_{k=1}^K \log(p(k)) \cdot q'(k) \\ &= (1-\epsilon) \text{Loss}(q, p) + \epsilon \cdot \text{Loss}(u, p) \end{aligned}$$

On further mathematics, by rewriting $\text{Loss}'(q', p)$ in terms of the KL divergence of q' from p :

$$\begin{aligned} \mathcal{L} &= D_{KL}(q' || p) + H(q') \\ &= D_{KL}(q' || p) + \underbrace{H(q)}_{\text{fixed}} + \dots \end{aligned}$$

The entropy of ~~the~~ q' is simply some function/linear combination of entropy of μ and entropy of δ , both of which are fixed. Since $D_{KL} \geq 0 \forall q' \neq p$, we see that $q' = p$ is the minimal case, which happens

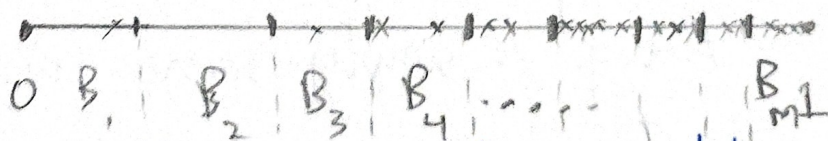
when $q_k = p_k = q_k = 1 - \alpha + \frac{\alpha}{K}(n)$. The point here being, that if the probability of any class were to go above $1 - \alpha + \frac{\alpha}{K}$, then the loss would increase, thereby driving max probability down.

ANALYSIS OF METRICS:

→ One of the major purposes of label smoothing is reducing overconfidence. This can be measured by a metric called Expected Calibration error:

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |acc(B_m) - conf(B_m)|$$

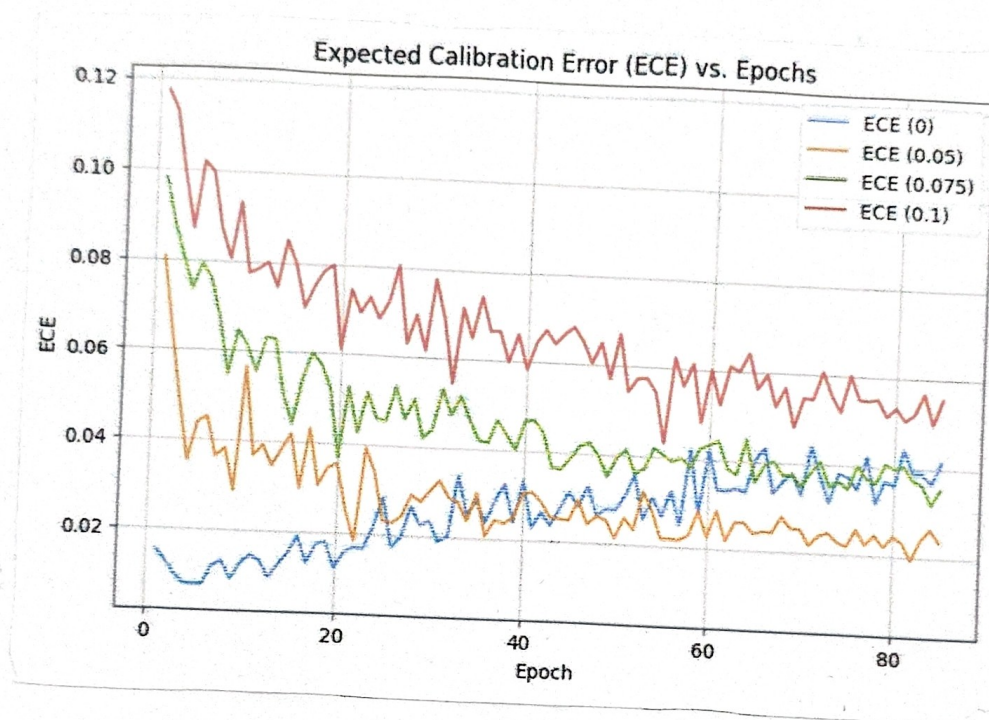
→ essentially, let us take all of the probability out - comes that the model predicts across multiple ~~training~~/testing examples and plot them on x axis,



now, we divide them into equally spaced bins (say M bins). each bin has a bin count B_m which is the number of samples in the m^{th} bin, out of a total n samples.

$\text{acc}(B_m)$ refers to the accuracy of the samples in the m^{th} bin (the chance they got that prediction right) and $\text{conf}(B_m)$ is the average probability reported in that bin.

→ A smaller ECE means a more well calibrated model.



→ Two different noticeable observations in this plot need to be addressed.

1. The ECE for labelsmoothed models moves down whereas that of baseline ($\alpha=0$) moves up, indicating overconfidence or overfitting. (this is done on validation set) so overfitting is indicated by ECE going up.)

2. The ECE for the models with label smoothing starts off higher ~~depen~~ and higher, ~~like~~ directly proportional to the α .

This is because label smoothing promotes more values into lower bins initially (B_3, B_4 etc) and since accuracy of everything is still pretty ~~bad~~ good there is a super high ECE. (80% after 1 epoch)

This phenomenon scales with α as more α promotes more values into smaller bins where they only come to about 60~70% while true accuracy is 83%.

The model without α , ($\alpha=0$ baseline) starts failing when its model predicts accuracy probabilities beyond the baseline ~~acc~~ accuracy.

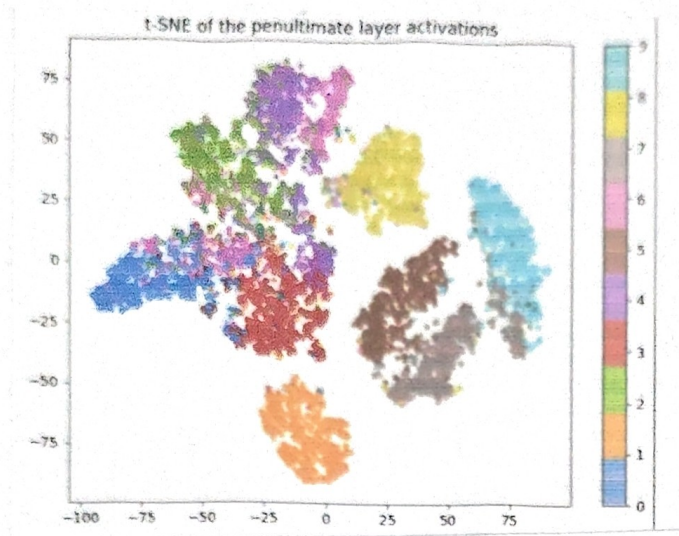
By epoch 50, we see $\alpha=0.05$ overtaking $\alpha=0$. (or rather becoming lower)

METRIC 2: T-sne on the Penultimate layer

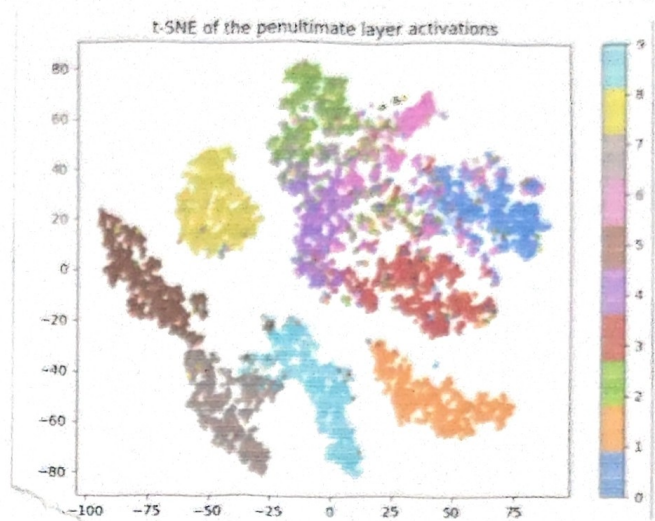
→ Notice how performing a tsne is significantly different from the method used in paper.

→ The method used in paper was doing an orthonormal projection on a plane with

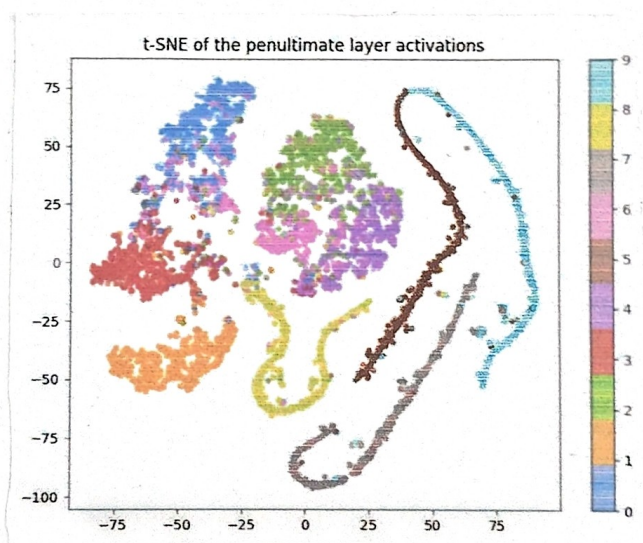
The centroids of 3 classes passing through.



T-sne on penultimate layer
for No label smoothing
($\alpha = 0$)



T-sne on penultimate layer
with label smoothing
($\alpha = 0.1$)



T-sne on label smoothing
extreme ($\alpha = 0.5$)

→ Points of observation :

1. The more the label smoothing, the more that the classes form an arc shape.

observing the blue, brown, olive green and grey classes, we see that label smoothing causes them to become arc shape objects that join together.

This actually means that the classes are "smushed" along a certain ^{straight} line in their ~~or~~ penultimate representations and are thus shown in T-sne to be line objects.

2. Observing other ~~classes~~ classes, we see that label smoothing causes them to separate out more.

The once homogeneous structure (red, pink, violet, green blue) became more of subgroups (separated).

→ Obviously, ~~0.5~~ $\alpha = 0.5$ is not an ideal value but goes to demonstrate the effects of label smoothing.

IMPROVEMENTS:

As the last disadvantage stated, a uniform distribution for $\mu(k)$ ruins interclass relationship.

Eg:- In the classification of Horse, cat, mouse, Titanic ship, for a true label of cat we equally soften amongst all classes when we know for a fact that horses, mouse should be favoured.

I propose using a dynamic $u(k, y)$ that depends on the true label.

There are two major implementations I can think of:

1. Using ordinary label smoothing, computing distances between centroids of each class, then using that as a relative $u(k, y)$ and running the model again.

We compute the centroid for each class by the formula:

ADVANTAGES AND DISADVANTAGES

So far in this report, we have already seen the effects that label smoothing has on a model.

The straight up advantages are:

1. Acts as a regularizer
2. Decreases overconfidence
3. Increases ~~over~~ interclass variances

What are the disadvantages?

1. Label smoothing causes representation degradation, i.e., the signal of output is disturbed.
2. As seen in the TSNE, it ~~introduces~~ causes feature representations to cluster too tightly along a certain line.
3. Interclass relationships are destroyed as non-target classes are given same weightage ~~over~~, disregarding interclass similarity.

$$u_c^{(t)} = \frac{1}{|D_c|} \sum_{i: y_i = c} \phi_\theta(x_i), \quad c \in \{1, \dots, C\}$$

D_c is the set of training with class as c .

u_c is the centroid of each class.

Then let us describe the similarity kernel as:

$$s_{cj}^{(t)} = \frac{\langle \mu_c^{(t)}, \mu_j^{(t)} \rangle}{\|\mu_c\| \|\mu_j\|}$$

Now, we describe the label smoothing as:

$$q_j^{(t)} = \begin{cases} 1 - \alpha & j = c \\ \alpha \cdot \frac{e^{s_{cj}}}{\sum_{k \neq c} e^{s_{ck}}} & j \neq c \end{cases}$$

Where α is the smoothing parameter.

2. We can update the centroid matrix dynamically during training. This method is too hard however to write mathematically! :(